Unifying LLMs with Knowledge Graphs for Reliable Chatbot Interactions.

Nguyen Tuan Anh[1]
Nanyang Technological University, Singapore
tuananh008@e.ntu.edu.sg

Abstract

Large Language Models (LLMs) have demonstrated remarkable language understanding and generation capabilities but often suffer from hallucinations and lack of interpretability. To address these issues, we present a framework that integrates LLMs with Knowledge Graphs (KGs) to improve the accuracy and explainability of the facts. This paper introduces a multistage architecture and a fine-tuning pipeline for SPARQL query generation. Compared to existing LLM baselines, our approach achieves more reliable factual grounding and provides clearer explanations through structured KG reasoning.

1 Introduction

1.1 Motivation

Large language models (LLMs) such as ChatGPT (OpenAI et al., 2023) and LLaMA (Touvron et al., 2023) have shown impressive performance in various NLP tasks such as text generation, summarization, and question answering. However, they suffer from limitations including hallucination and a black-box nature that hinders interpretability. These models often generate factually incorrect information due to their reliance on parametric memory and lack of structured real-time knowledge (Petroni et al., 2019; Ji et al., 2023).

In contrast, Knowledge Graphs (KG) (Hogan et al., 2021) offer structured, symbolic representations of facts in the form of entity-relation triples. They enable verifiable reasoning and provide an interpretable and up-to-date factual basis. Recent work has highlighted the promise of integrating KG with LLM to address the issues of factuality and explainability (Chen et al., 2023). This integration can help LLMs retrieve accurate information, reduce hallucinations, and improve user trust in AI systems.

1.2 Problem Statement

Despite the success of LLM in language understanding, they struggle with factual grounding and transparency. Their internal representations do not provide verifiable reasoning paths, and they often fail to maintain factual consistency, especially in knowledge-intensive tasks. This highlights the need for external knowledge integration to improve reliability and interpretability.

1.3 Contribution Summary

This paper proposes a novel *Unifying-LLM-KG Framework*, which integrates LLMs with structured querying from Knowledge Graphs (KGs) to improve factual grounding and interpretability. Key contributions include:

- A unified architecture combining KG-enhanced LLMs through modular coordination.
- An iterative query refinement strategy that improves SPARQL generation through trial-and-error feedback.
- Comparative evaluation against a strong LLM baseline (DeepSeek-V3) using the MCQA dataset.

1.4 Paper Organization

Section 2 reviews background work on LLMs and KGs. Section 3 outlines our architecture and system components. Section 4 covers experimental setup, results and discussions. Section 5 presents our chatbot application. Section 6 discusses future directions.

2 Background and Related Work

2.1 Overview of Large Language Models

Large Language Models (LLMs) such as GPT-4 (OpenAI et al., 2023), LLaMA (Touvron et al., 2023), and Phi-2 (Gunasekar et al., 2023) are based on transformer architectures and trained on

large-scale text corpora to model language patterns. These models demonstrate state-of-the-art performance across tasks like text generation, summarization, and question answering. However, their reliance on static parametric memory limits their ability to provide factually grounded responses (Petroni et al., 2019; Ji et al., 2023). Moreover, their internal mechanisms lack interpretability, making it difficult to trace how conclusions are reached, especially in critical applications such as medicine, finance, or law (He et al., 2024; Lloyd and Patel, 2024; Research, 2024).

2.2 Knowledge Graphs

Knowledge Graphs (KGs) are structured representations of entities and their relationships, typically expressed as triples in the form of *(subject, predicate, object)* (Hogan et al., 2021). Standards such as RDF (Resource Description Framework) (rdf, 2014) and SPARQL (SPARQL Protocol and RDF Query Language) (spa, 2013) facilitate semantic querying and reasoning over these graphs. Graph databases like GraphDB (gra, 2025), Neo4j (neo, 2025), and Amazon Neptune (ama, 2025) enable efficient storage, retrieval, and inference.

2.3 KG-Enhanced LLMs

To overcome LLMs' limitations in factuality and interpretability, recent work has explored integrating external knowledge from KGs. This hybridization generally falls into three categories (Chen et al., 2023):

- **Pre-training-based:** Knowledge from KGs is injected during model pre-training via entity linking or relation-aware attention.
- **Inference-based:** LLMs access KGs at inference time, either by retrieving relevant triples or by generating structured queries.
- **Interpretability-focused:** KGs are used to post-process or explain model outputs by grounding them in explicit knowledge.

2.4 Prior Work

Several works have explored combining LLMs with KGs for robust reasoning. **RoG (Reasoning over Graphs)** (Xu et al., 2022) introduced a planning-retrieval-generation framework that guides LLMs using predefined graph reasoning paths. While RoG improves factual grounding, it heavily relies on annotated path templates and lacks scalability.

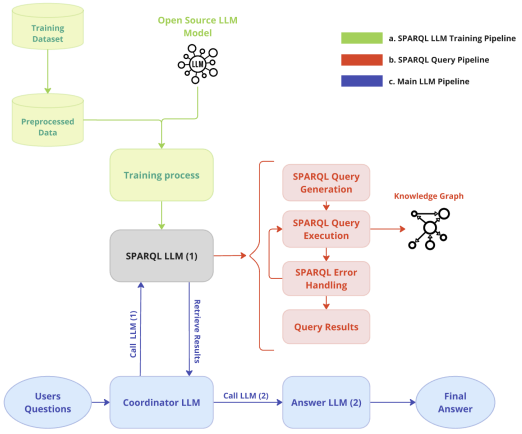


Figure 1: Overall architecture of the Unifying-LLM-KG framework.

ChatKBQA (Zhang et al., 2023) employs a generate-then-retrieve pipeline: it first generates a structured SPARQL query based on user input and then retrieves the answer from the KG. Although it improves precision, this design merges reasoning and query generation within a single LLM, limiting modularity and transparency.

Our approach addresses these limitations by separating query generation, execution, and response formation into modular components. It leverages fine-tuned open-source LLMs to generate SPARQL queries that are dynamically adjusted to domain-specific ontologies, enabling greater adaptability and interpretability in chatbot applications.

3 Methodology: Unifying-LLM-KG Framework

The proposed framework integrates Large Language Models (LLMs) with a structured Knowledge Graph (KG) backend to enable accurate, interpretable, and traceable chatbot responses. As shown in Figure 1, the system consists of three main components:

1. **SPARQL LLM Training Pipeline** – prepares a fine-tuned LLM for translating natural language questions into SPARQL queries.
2. **SPARQL Query Pipeline** – executes and validates queries against the KG, with automatic correction via LLM.
3. **Main LLM Pipeline** – orchestrates the question-answering flow, bridging user input with structured knowledge and fluent output.

3.1 SPARQL LLM Training Pipeline

To enable accurate and resource-efficient SPARQL query generation, we fine-tune open-source LLMs (e.g., LLaMA, Qwen, Phi) using a parameter-efficient approach. Instead of full-model fine-tuning, we adopt Low-Rank Adaptation (LoRA) to update a small subset of weights, optimizing for both memory and generalization. Figure 2 illustrates the overall pipeline.

Step 0: Model Initialization. We begin by loading a pretrained LLM with 4-bit quantization to reduce memory usage and improve training throughput. Given a full-precision matrix $W \in R^{m \times n}$, quantization applies:

$$Q(W) = S \cdot \text{round}(W/S)$$

where S is a scaling factor. This transformation compresses the model to 25% of its original size, enabling low-bit matrix multiplications without sacrificing stability.

Step 1: Data Preprocessing. Training data consists of system-user-assistant message triples stored in JSON format. Each instance is serialized into token sequences using a chat-style prompt template. Let $x = (q, r)$ denote a query-response pair; tokenization yields $T(x) = [t_1, \ldots, t_n]$. Messages are formatted into role-based structures aligned with the tokenizer's expected input to preserve context.

Step 2: Fine-Tuning with LoRA. We inject low-rank matrices into attention and feedforward projections:

$$W' = W + \Delta W, \quad \Delta W = AB$$

where $A \in R^{m \times r}$ and $B \in R^{r \times n}$ with $r \ll \min(m, n)$. Only the adapter weights (A, B) are trainable, significantly reducing parameter updates. Training uses the AdamW optimizer with weight decay and cosine learning rate scheduling:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$$

Step 3: Loss Computation. We minimize token-level cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{V} y_{i,j} \log p_{i,j}$$

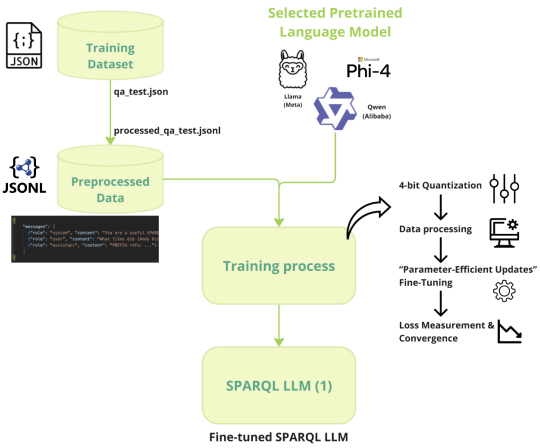


Figure 2: SPARQL LLM training pipeline illustrating 4-bit quantization, structured message formatting, and LoRA-based adaptation.

Algorithm 1 SPARQL LLM Training Pipeline

Require: Pretrained LLM M, tokenizer T, dataset D, LoRA config P
Ensure: Fine-tuned model M'

$M, T \leftarrow$ LoadModel(`name`, 4-bit)
$M' \leftarrow$ ApplyLoRA($M, P = \{$`q,k,v,o,gate,up,down_proj`$\}$)
$D \leftarrow$ LoadDataset("`train.jsonl`")
$D \leftarrow$ FormatData(D, `chat_template`)
SetHyperparams(batch $= 2$, grad_acc $= 4$, lr $=$ 2e–4)
Opt $\leftarrow$ AdamW(`weight_decay` $= 0.01$)
Trainer $\leftarrow$ Train(M', D, Opt)
$\mathcal{L} \leftarrow$ ComputeLoss(M', D)
return M'

where $y_{i,j}$ is the one-hot label and $p_{i,j}$ is the model-predicted probability for token j. Loss convergence is monitored to ensure stable optimization.

3.2 SPARQL Query Pipeline

This component handles runtime SPARQL execution and robust query validation. It ensures that user intents are faithfully transformed into executable KG queries and that errors are addressed automatically.

Execution Layer

The system connects to GraphDB, dispatches the query, and parses results based on response type (e.g., `ASK`, `SELECT`).

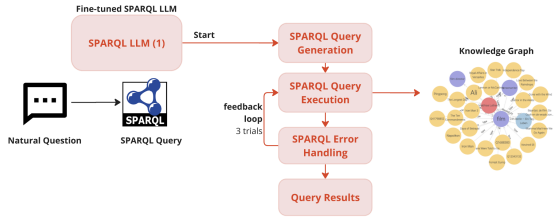


Figure 3: SPARQL query pipeline with runtime execution and LLM-based repair.

Algorithm 2 SPARQL Query Execution

Require: SPARQL query Q, knowledge graph G
Ensure: Query result R

$conn \leftarrow \text{OPENGRAPHDB}(G)$
$store \leftarrow \text{CREATESPARQLSTORE}(conn)$
$graph \leftarrow \text{INITGRAPH}(store)$
try: $R \leftarrow \text{GRAPH.QUERY}(Q)$
if R.type == `ASK` **then return** Boolean(R)
else if R.type == `SELECT` **then return** Parsed rows
elsereturn Graph result
except: $\text{HANDLEERROR}(Q, e)$

LLM-Guided Query Repair

To handle malformed queries or ontology mismatches, we introduce a retry mechanism powered by an instruction-tuned LLM. When errors are detected, the system dynamically regenerates the query based on the original question and error message.

Algorithm 3 SPARQL Query Retry with LLM Repair

Require: Query Q, KG G, retry cap N
Ensure: Valid result R or exception

$retries \leftarrow 0$, $error \leftarrow$ None
while $retries < N$ **do**
  try: $R \leftarrow \text{EXECUTEQUERY}(Q, G)$ **return** R
  except:
    $error \leftarrow \text{STR}(e)$
    $Q' \leftarrow \text{LLMFIX}(Q, error, question)$
    $Q \leftarrow Q', retries \leftarrow retries + 1$
raise Exception("Query cannot be repaired")

LLM Prompt:

> *"You are a helpful SPARQL assistant. Given a question, a failed query, and an error message, generate a corrected SPARQL query."*

This approach enables robust, rule-free query recovery with minimal tuning.

3.3 Main LLM Pipeline

This component orchestrates the overall flow between user intent, structured retrieval, and fluent response generation. It consists of three specialized modules:

Coordinator LLM. Routes queries, decides on KG retrieval, invokes the SPARQL LLM and Answer LLM, and logs conversation state.
SPARQL LLM. Translates natural questions into executable SPARQL queries and retrieves structured answers from GraphDB.
Answer LLM. Converts retrieved facts into fluent, accurate, and contextually grounded responses.

Algorithm 4 Main Pipeline Inference Logic

Require: User query Q, fine-tuned model M, KG G
Ensure: Natural language answer R

if Q is empty **then return** "No input provided"
Append Q to `messages`; display "User: " $+Q$
$Q_{SP} \leftarrow \text{GENERATESPARQL}(Q, M)$
Display "SPARQL query: " $+Q_{SP}$
$R_G \leftarrow \text{EXECUTEQUERY}(Q_{SP}, G)$
$F_R \leftarrow \text{FORMATRESULTS}(R_G)$
$R \leftarrow \text{GENERATEANSWER}(Q, F_R)$
Append R to `messages`; display "Assistant: " $+R$
return R

4 Experiments

This section presents the experimental results of the proposed SPARQL LLM pipeline, highlighting its effectiveness in generating accurate structured queries across diverse question types. We benchmark our approach against a strong baseline (DeepSeek-V3) and evaluate multiple fine-tuned open-source models.

4.1 Benchmark Dataset

We evaluate our system on the MCQA benchmark (Hoang et al., 2024), a large-scale Movie Complex Question Answering dataset designed to test reasoning over knowledge graphs. MCQA contains a total of 141,979 questions grounded in a structured knowledge base derived from iMKG, an RDF-style knowledge graph built using entities

Category	Question Type	Example
Comparative	compare_directors _num_movies	Which director has helmed more films, [director] or [director]?
Count	actor_year _num_movies	How many movies did [actor] appear in during [year]?
Intersection	director_year_movie	Which actors were in films directed by [Director] in [Year]?
Multi-hop, Constraint	movie_to_actor _constraint_movie	Who starred in both [movie] and [movie]?
Yes/No	yesno_actor_genre	Has [actor] appeared in any [genre] films?

Table 1: Examples of question types in the MCQA dataset.

from WikiMovies (wik) and Wikidata (Wikidata contributors, 2025).

The dataset consists of 110,409 training examples and 31,570 test instances, each comprising a natural language question, a corresponding SPARQL query, and gold answer entities. Questions span 83 types and are generated from 4,135 distinct templates. These include a diverse mix of comparative, constraint-based, counting, intersectional, multi-hop, and yes/no question formats.

For example, a question such as *"Can you tell me which films have the same screenwriter as* `It Came from Beneath the Sea` *and were released in 1952?"* involves reasoning over shared attributes and temporal constraints. Table 1 summarizes representative question categories and examples.

This diversity makes MCQA a robust benchmark for evaluating both the semantic understanding and logical inference capabilities of LLMs integrated with knowledge graphs.

4.2 Evaluation Metrics

We assess model performance based on the correctness of the retrieved answers from generated SPARQL queries. Instead of comparing entire query strings, we evaluate each predicted answer element against the gold standard using set-based comparison over answer IDs.

Evaluation Setup.

- **Ground Truth Results**: The expected set of answer entities provided in the annotated dataset.
- **Predicted Results**: The answer entities retrieved by executing the SPARQL query generated by the model.

Evaluation Metrics. We use three widely accepted metrics, defined as follows:

- **True Positives (TP)**: Correct answers that are present in both predicted and ground truth sets.
- **False Positives (FP)**: Incorrect answers that appear in the prediction but not in the ground truth.
- **False Negatives (FN)**: Missed correct answers that are in the ground truth but not retrieved.

Final Metrics. From the above definitions, we compute the following:

- **Precision** = $\frac{\text{TP}}{\text{TP+FP}}$: Measures the proportion of retrieved answers that are actually correct.
- **Recall** = $\frac{\text{TP}}{\text{TP+FN}}$: Measures the proportion of correct answers that were successfully retrieved.
- **F1 Score** = $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision+Recall}}$: Harmonic mean of precision and recall, capturing their trade-off.

4.3 Models and Training Setup

We compare one baseline LLM (DeepSeek-V3) and three instruction-tuned 3B-scale models:

- **DeepSeek-V3** (baseline): Accessed via API; used also as Coordinator and Answer LLM.
- **Llama-3.2-3B-Instruct**, **Qwen2.5-3B-Instruct**, **Phi-3.5-mini-Instruct**: Trained on 4.1k MCQA samples.

Training used LoRA with parameter-efficient fine-tuning. Inference was conducted using a single NVIDIA RTX A5000. Table 2 summarizes hyperparameters.

Model	Steps	Batch	Trainable	Params
Llama-3.2	518	8	24.3M	3.0B
Qwen2.5	518	8	29.9M	3.1B
Phi-3.5	129	32	29.8M	3.8B

Table 2: Training setup for SPARQL LLM models.

4.4 Results

4.4.1 Overall Performance

Table 3 presents the overall performance of the SPARQL LLMs across 31,570 test queries. The instruction-tuned models significantly outperform the baseline, with DeepSeek-V3 lagging behind in all metrics (Precision: 33.81%, Recall: 24.14%, F1: 28.16%). Among the fine-tuned models, **Phi-3.5-mini** achieves the highest overall performance, attaining a Precision of 95.56% and F1 Score of 93.92%. **Qwen2.5-3B** and **Llama-3.2-3B** both exhibit strong Recall (above 91%), though Qwen2.5 surpasses Llama in Precision and F1.

Model	Prec.	Recall	F1	Time
DeepSeek-V3	33.81	24.14	28.16	98:57:57
Llama-3.2	87.17	**91.60**	89.33	23:59:01
Qwen2.5	92.39	91.46	91.92	30:39:01
Phi-3.5	**95.56**	91.13	**93.92**	72:32:14

Table 3: Performance comparison of SPARQL LLMs across precision, recall, F1, and inference time.

4.4.2 Per-Type Breakdown

Figures 4 and 5 illustrate the per-question-type performance of **Phi-3.5-mini-instruct** and **DeepSeek-V3**, respectively.

Phi-3.5 demonstrates exceptional precision across a wide range of multi-hop and compositional question types. In particular, it achieves over 99% precision on several complex categories such as `movie_to_actor_to_movie_...genre` and `...constraint_writer`, indicating robust generalization on relation-chained queries.

In contrast, **DeepSeek-V3** consistently underperforms. Even its best-performing type—`yesno_movie_language`—only reaches 81.25% precision, while more structured types such as `year_num_movies` or `actor_year_num_movies` show near-zero performance on both precision and recall. This highlights DeepSeek's limitations in handling logical constraints, numerical reasoning, and multi-hop inference.

Overall, Phi-3.5 outperforms DeepSeek-V3 not only in aggregate metrics but also across nearly all fine-grained question categories.

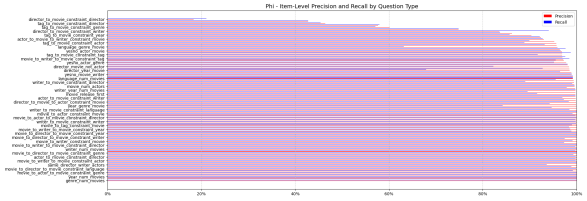


Figure 4: Precision and recall by question type for Phi-3.5-mini-instruct.

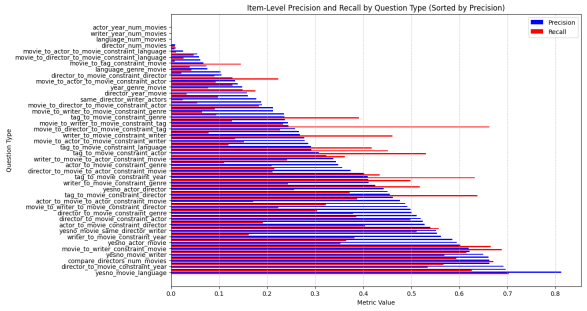


Figure 5: Precision and recall by question type for DeepSeek-V3.

4.5 Root Cause Analysis

We conducted a qualitative analysis of model errors to understand common failure modes in SPARQL query generation. Several recurring issues were identified:

- **Relation mislabeling:** Models often confuse semantically related predicates (e.g., `written_by` vs. `directed_by`), leading to incorrect query semantics.
- **Logical structure errors:** Multi-entity conditions are sometimes improperly represented, with logical constraints not applied to the same subject, resulting in unintended disjunction or exclusion.
- **Entity encoding failures:** Names with diacritics or special characters frequently cause mismatches in entity linking due to tokenization or normalization errors.
- **Temporal misalignment:** Time-based constraints are occasionally misapplied to the wrong relation or entity, reducing retrieval accuracy.

Figure 6 shows representative examples of these errors, comparing generated and corrected SPARQL queries.

✗ Generated (Incorrect) SPARQL Query:	✓ Correct SPARQL Query:
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> SELECT DISTINCT ?uri ?l WHERE { ?uri ?rel1 ?e1; ?rel2 ?e2; rdfs:label ?l. ?rel1 rdfs:label "directed_by". ?e1 rdfs:label "Arthur Hotaling". ?rel2 rdfs:label "written_by". ?e2 rdfs:label "Oliver Hardy".}	PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> SELECT DISTINCT ?uri ?l WHERE { ?uri ?rel1 ?e1; ?rel2 ?e2; rdfs:label ?l. ?rel1 rdfs:label "directed_by". ?e1 rdfs:label "Arthur Hotaling". ?rel2 rdfs:label "directed_by". ?e2 rdfs:label "Oliver Hardy".}

Figure 6: Examples of common SPARQL errors: predicate mislabeling, logical inconsistency, and encoding mismatches.

4.6 Discussion

Efficiency Trade-offs. **Phi-3.5** achieved the highest F1 score but also incurred the longest inference time. In contrast, **Llama-3.2** and **Qwen2.5** provided strong accuracy with lower latency. **DeepSeek-V3** was the slowest due to API overhead, including request latency and response parsing.

Model Behavior. Fine-tuned models excel at structured, multi-hop queries, but still struggle with constraint-heavy and temporal reasoning:

- Precision is highest for clean relational patterns.
- Recall drops in cases involving ambiguous or time-based logic.

Improving Precision and Recall. Performance can be further enhanced through:

- Schema-aware prompts with entity and relation hints.
- Multi-turn prompting for complex queries.
- Larger models for deeper reasoning ability.

Why DeepSeek as Coordinator/Answer LLM? DeepSeek offers fluent response generation and reliable query coordination via API. Its efficiency and cost-effectiveness make it ideal for parsing structured outputs and orchestrating multi-component pipelines.

5 Application: LLM-KG Integration for Chatbot QA

We implement **KGChat**, a locally hosted prototype that demonstrates the integration of structured knowledge querying within a conversational interface. Built on the Unifying LLM-KG framework, KGChat enables side-by-side comparison between a base language model and a SPARQL-augmented, fine-tuned alternative.

5.1 Technical Implementation

KGChat is deployed entirely on local infrastructure using Streamlit for the interface, FastAPI for backend inference, and GraphDB for structured query execution. The system is organized into two parallel response pipelines:

- **Thread 1 (Unifying LLM-KG):** User queries are routed to a FastAPI endpoint running on a GPU VM. A selected fine-tuned model (LLaMA-3B, Qwen2.5-3B, or Phi-3.5) generates a SPARQL query, which is executed against a local GraphDB instance. The resulting triples are passed to DeepSeek-V3, which serves as the Answer LLM to verbalize the results.
- **Thread 2 (Baseline LLM):** The same input is sent directly to DeepSeek-V3 via its API, producing a purely generative response without KG integration.

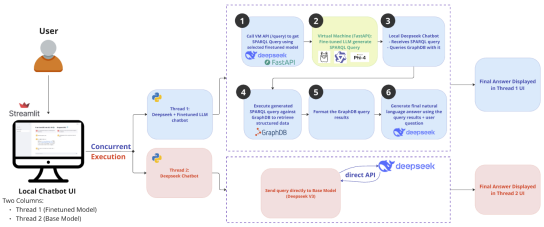


Figure 7: System workflow of KGChat. Thread 1 executes structured retrieval and verbalization; Thread 2 returns a base model response.

The backend includes the following components:

- **FastAPI server:** Hosts fine-tuned LLMs and manages SPARQL generation.
- **GraphDB endpoint:** Executes queries and returns structured knowledge triples.
- **DeepSeek-V3:** Acts as both a verbalizer (Thread 1) and baseline model (Thread 2).
- **ThreadPoolExecutor:** Enables asynchronous execution of both response threads.
- **Streamlit interface:** Presents both outputs side-by-side for qualitative comparison.

To support transparency, generated SPARQL queries are optionally shown before response generation.

5.2 Qualitative Evaluation

We qualitatively evaluate KGChat on select MCQA test questions, comparing fine-tuned structured reasoning models against a baseline LLM.

The SPARQL-based pipeline consistently produces more accurate and interpretable responses, especially for multi-hop and constraint queries.

Case 1: Unifying LLM-KG outperforms base model

- *Q1: How many films did Dhimitër Anagnosti direct?*
  KGChat (LLaMA): 14 **DeepSeek:** 5
- *Q2: What movies share the same screenwriter as* `Paris When It Sizzles` *and are classified as mystery?*
  KGChat (Qwen): The Lady Vanishes
  DeepSeek: The Manchurian Candidate
- *Q3: Can you tell me about any movies Harold Ramis starred in that fall under the genre of comedy film?*
  KGChat (Phi): [Ghostbusters, The Last Kiss, National Lampoon's Vacation, High Fidelity, Drunk Stoned Brilliant Dead, As Good as It Gets, Airheads, Groundhog Day, Ghostbusters II, Orange County]
  DeepSeek: [Caddyshack, Stripes, Meatballs, Ghostbusters, Groundhog Day]

Case 2: Both models succeed

- *Q4: What fantasy movie stars Catinca Untaru?*
  Both: The Fall
- *Q5: Was* `Three Heroes and the King of the Sea` *released in Russian?*
  Both: Yes

These examples highlight the benefit of integrating symbolic knowledge into generative pipelines for improving factual accuracy and response fidelity.

6 Conclusion and Future Work

We proposed and validated a robust framework integrating LLMs with KGs, significantly enhancing factual grounding and interpretability in chatbot interactions. Through fine-tuning and structured querying, we demonstrated clear advantages over baseline methods in both quantitative metrics and qualitative evaluations.

Future research directions include:

- Adapting the framework to specialized domains such as finance, healthcare, and legal applications, where factual accuracy and explainability are crucial.
- Expanding our approach to larger-scale KGs and broader domains beyond movies.
- Investigating real-time incremental KG updates to ensure chatbot responses remain current.
- Enhancing query repair mechanisms using reinforcement learning to adaptively refine SPARQL queries.

Additionally, extending this methodology to multimodal knowledge sources and incorporating user feedback for dynamic query refinement represent promising areas for future development.

Our findings reinforce the value of structured knowledge integration in enhancing LLM capabilities, paving the way for more reliable, interpretable, and trustworthy AI-driven conversational systems.

References

Wikimovies dataset. https://paperswithcode.com/dataset/wikimovies. Accessed: 2025-05-05.

2013. Sparql 1.1 query language. https://www.w3.org/TR/sparql11-query/.

2014. Rdf 1.1 concepts and abstract syntax. https://www.w3.org/TR/rdf11-concepts/.

2025. Amazon neptune. https://aws.amazon.com/neptune/.

2025. Graphdb knowledge graph database. https://www.ontotext.com/products/graphdb/.

2025. Neo4j graph database. https://neo4j.com/.

Hongwei Chen, Dong Liu, Yichi Wang, Sen Wu, and Qiaoling Liu. 2023. Unifying large language models and knowledge graphs: A roadmap. *arXiv preprint arXiv:2306.08302*.

Suriya Gunasekar and 1 others. 2023. Phi-2: The surprising power of small language models. *Microsoft Research*. https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/.

Jinyan He, Fei Liu, and Haoran Zhang. 2024. Opportunities and challenges for llms in clinical decision support. *NPJ Digital Medicine*.

Lily Hoang, Fiona Liausvia, Liu Yan, and Thanh-Son Nguyen. 2024. Semi-automated construction of complex knowledge base question answering dataset using large language model. In *Proceedings of the European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD)*.

Aidan Hogan, Eva Blomqvist, Michael Cochez, and 1 others. 2021. Knowledge graphs. *ACM Computing Surveys (CSUR)*, 54(4):1–37.

Zhengbao Ji, Nan Shao, Zhiyong Wu, Tuo Zhao, and Zhifang Sui. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys (CSUR)*, 55(12):1–38.

Jonathan Lloyd and Arjun Patel. 2024. Legal transparency and accountability in ai-powered finance. *Reuters Legal*. Available online at https://www.reuters.com/legal/transactional/legal-transparency-ai-finance-facing-accountability-dilemma-digital-decision-2024-03-01/.

OpenAI and 1 others. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. 2019. Language models as knowledge bases? pages 2463–2473.

Sapien AI Research. 2024. Reducing hallucinations in large language models. Available at https://www.sapien.io/blog/reducing-hallucinations-in-llms.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, and 1 others. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Wikidata contributors. 2025. Wikidata: A free knowledge base. https://www.wikidata.org/. Accessed: 2025-05-05.

Haoyu Xu, Chen Liang, Nikhil Gupta, and 1 others. 2022. Reasoning over graphs for explainable qa. In *Proceedings of ACL*.

Wei Zhang, Lei Wang, Shilin Duan, and 1 others. 2023. Chatkbqa: Improving conversational kgqa with context-aware query generation. In *Proceedings of EMNLP*.